

Reto Técnico FullStack + Data Analytics + ETL + Machine Learning

Plataforma Inteligente de Analítica Clínica para Detección de Riesgo Médico



Descripción General del Reto

Una reconocida IPS llamada HealthAnalytics IPS requiere desarrollar una plataforma web inteligente capaz de procesar grandes volúmenes de datos clínicos mediante procesos ETL, analítica de datos y modelos básicos de Machine Learning.

La organización actualmente presenta problemas relacionados con:

- Mala calidad de datos clínicos.
- Duplicidad de pacientes.
- Registros médicos inconsistentes.
- Falta de indicadores clínicos.
- Dificultad para detectar pacientes de alto riesgo.
- Ausencia de automatización en análisis predictivo.

El objetivo del sistema será automatizar el procesamiento de información médica y apoyar la toma de decisiones clínicas mediante analítica predictiva.

Objetivo General

Diseñar, desarrollar e implementar una aplicación web FullStack utilizando Python, Django, Pandas y MySQL/PostgreSQL que permita:

- Ejecutar un proceso ETL completo.
- Limpiar y transformar datos clínicos.
- Analizar métricas médicas.
- Detectar pacientes críticos.
- Predecir riesgos de enfermedades.
- Visualizar información mediante dashboards interactivos.



Contexto del Problema Real

La IPS recibe diariamente miles de registros clínicos provenientes de:

- Consultas externas.
- Laboratorios.
- Urgencias.
- Hospitalización.
- Sistemas externos.

Los datos llegan con múltiples inconsistencias:

- Campos vacíos.
- Valores incorrectos.
- Duplicados.
- Diagnósticos mal escritos.
- Datos fuera de rango.
- Tipos de datos inválidos.

La empresa necesita una solución tecnológica capaz de:

1. Centralizar información.
2. Estandarizar datos.
3. Detectar anomalías médicas.
4. Clasificar pacientes por riesgo.

5. Generar alertas tempranas.
6. Obtener estadísticas clínicas.
7. Predecir enfermedades.

Nivel del Reto

- Intermedio – Avanzado.
- Orientado a:
 - Desarrolladores FullStack.
 - Analistas de Datos.
 - Aprendices SENA.
 - Estudiantes de Ingeniería de Sistemas.
 - Especialistas en IA y Ciencia de Datos.

Tecnologías Obligatorias

Backend

- Python 3.12+
- Django
- Django REST Framework
- Pandas
- NumPy
- Scikit-Learn
- Celery (opcional)
- JWT Authentication

Base de Datos

Debe seleccionarse una:

- PostgreSQL
- MySQL

Frontend

- HTML5

- CSS3
- Bootstrap 5
- JavaScript
- Chart.js o Plotly

Herramientas Complementarias

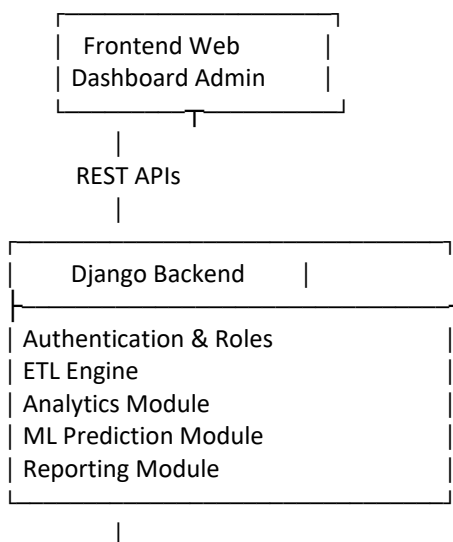
- Docker (opcional)
- Git/GitHub
- Swagger/OpenAPI
- Postman

Arquitectura Esperada

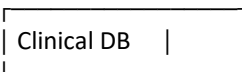
La solución debe seguir una arquitectura desacoplada basada en:

- Backend API REST.
- Frontend administrativo.
- Módulo ETL independiente.
- Módulo de analítica.
- Motor de Machine Learning.

Arquitectura Propuesta



PostgreSQL/MySQL



Objetivo Funcional del Sistema

El sistema deberá:

- Generar datasets clínicos simulados.
- Ejecutar ETL automáticamente.
- Clasificar pacientes por riesgo.
- Mostrar dashboards clínicos.
- Ejecutar predicciones médicas.
- Generar reportes.
- Administrar usuarios y roles.

Dataset Clínico Simulado

Cantidad de Registros

Generados automáticamente: 1800 registros clínicos simulados

El archivo Excel entregado con el dataset clínico simulado fue generado aleatoriamente e Incluye:

- 1800 registros base.
- Registros duplicados intencionales.
- Valores nulos.
- Tipos de datos incorrectos.
- Valores atípicos.
- Errores ortográficos simulados.
- Variables clínicas completas para el proceso ETL y Machine Learning.

Variables Obligatorias

Campo

Tipo

<i>id_paciente</i>	Integer
<i>nombres</i>	String
<i>apellidos</i>	String
<i>edad</i>	Integer
<i>sexo</i>	String
<i>peso</i>	Float
<i>altura</i>	Float
<i>IMC</i>	Float
<i>presión_sistólica</i>	Integer
<i>presión_diastólica</i>	Integer
<i>frecuencia_cardiaca</i>	Integer
<i>glucosa</i>	Float
<i>colesterol</i>	Float
<i>saturación_oxígeno</i>	Float
<i>temperatura</i>	Float
<i>antecedentes_familiares</i>	Boolean
<i>fumador</i>	Boolean
<i>consumo_alcohol</i>	Boolean
<i>actividad_física</i>	String
<i>diagnóstico_preliminar</i>	String
<i>riesgo_enfermedad</i>	String
<i>fecha_consulta</i>	Date

Requerimiento Crítico del Dataset

El dataset incluye errores reales.

Inconsistencias Obligatorias

Valores Nulos

Ejemplos:

glucosa = NULL
peso = NULL

Duplicados

Pacientes repetidos.

Tipos Incorrectos

edad = "Treinta"
presión_sistólica = "Alta"

Valores Atípicos

peso = 420 kg
temperatura = 28°C

Errores Ortográficos

"hipertensión"
"hipertencion"
"hipertensión"

Proceso ETL Obligatorio

1. EXTRACT — Extracción

Requerimientos

El sistema debe:

- Generar automáticamente el dataset.
- Exportar CSV.
- Leer archivos CSV externos.
- Registrar fuente de datos.

Evidencias Esperadas

- Archivo CSV generado.
- Logs de extracción.
- Tiempo de ejecución.

2. TRANSFORM — Transformación

Reglas Obligatorias

Limpieza de Datos

- Eliminar duplicados.
- Corregir inconsistencias.
- Validar rangos clínicos.

Conversión de Tipos

Ejemplo:

edad -> int
glucosa -> float

Normalización

Estandarizar:

- Diagnósticos.
- Sexo.
- Variables categóricas.

Tratamiento de Nulos

Aplicar:

- Media.
- Moda.
- Mediana.
- Reglas clínicas.

Clasificación de Riesgo

Riesgos esperados

- Bajo
- Medio
- Alto
- Crítico

Cálculo Automático de IMC

El sistema debe calcular:

$$IMC = \frac{\text{peso}}{\text{altura}^2}$$

Clasificación Clínica

Ejemplo:

<i>IMC</i>	<i>Clasificación</i>
<18.5	Bajo peso
18.5–24.9	Normal
25–29.9	Sobrepeso
>30	Obesidad

3. LOAD — Carga

El sistema debe:

- Insertar datos limpios en BD.
- Registrar logs ETL.
- Mostrar trazabilidad.
- Generar histórico ETL.

Machine Learning

Objetivo

Predecir probabilidad de enfermedad.

Modelos Obligatorios

Implementar mínimo uno:

- Regresión logística.
- Árbol de decisión.
- Random Forest.

Variables Predictoras

Ejemplo:

- IMC
- Edad
- Glucosa
- Colesterol
- Presión arterial
- Frecuencia cardiaca
- Fumador

Flujo ML

Dataset Limpio



Preprocesamiento



Entrenamiento



Evaluación



Predicción



Dashboard

Métricas Obligatorias

El sistema debe mostrar:

- Accuracy
- Precision
- Recall
- F1-Score
- Matriz de confusión

Fórmula Accuracy

$$\text{Accuracy} = \frac{\text{Predicciones Correctas}}{\text{Total Predicciones}}$$

Requerimientos de Analítica de Datos

Implementar:

Estadística Descriptiva

- Media
- Mediana
- Moda
- Desviación estándar

KPIs Médicos

- Pacientes críticos.
- Pacientes hipertensos.
- Pacientes diabéticos.
- Pacientes fumadores.
- Riesgo promedio.

Segmentación de Pacientes

Por:

- Edad
- Sexo
- Riesgo
- Diagnóstico
- IMC

Detección de Pacientes Críticos

Ejemplo:

Presión sistólica > 180
Glucosa > 300
Saturación < 85

Frontend y Dashboard

El sistema debe incluir:

Dashboard Administrativo

Visualizar:

- Total registros.
- Riesgo clínico.
- KPIs.
- Predicciones.
- Procesos ETL ejecutados.

Gráficas Obligatorias

- Barras.
- Líneas.
- Torta.

- Heatmaps.
- Tendencias.

Funcionalidades Obligatorias

Seguridad

Implementar:

- Login.
- JWT.
- Roles.
- Permisos.

Roles del Sistema

Rol	Funciones
Administrador	Gestión completa
Médico	Visualización clínica
Analista	ETL y analítica

Funcionalidades Extra

Subida Manual CSV

Permitir:

- Cargar nuevos datasets.
- Validar formato.
- Ejecutar ETL automáticamente.

Exportación

Exportar:

- PDF
- Excel
- CSV

Historial ETL

Guardar:

- Fecha.
- Usuario.
- Registros procesados.
- Tiempo ejecución.
- Estado.

APIs REST Obligatorias

Ejemplos

/api/auth/login/
/api/pacientes/
/api/etl/run/
/api/reportes/
/api/predicciones/
/api/dashboard/kpis/

Estructura Esperada del Proyecto

```
healthcare-etl-platform/  
|  
├── backend/  
│   ├── config/  
│   ├── apps/  
│   │   ├── authentication/  
│   │   ├── etl/  
│   │   ├── analytics/  
│   │   ├── ml/  
│   │   └── dashboard/  
└──
```

```
| | └─ reports/
| |
| └─ requirements.txt
|   └─ manage.py
|
└─ frontend/
    └─ templates/
        └─ static/
            └─ dashboard/
└─ datasets/
└─ docs/
└─ docker/
└─ README.md
```

Requerimientos No Funcionales

Escalabilidad

La solución debe soportar:

- Miles de registros.
- Procesamiento concurrente.
- Múltiples usuarios.

Seguridad

Implementar:

- Validación de datos.
- Protección CSRF.
- Sanitización.
- Roles y permisos.

Buenas Prácticas

Aplicar:

- Arquitectura limpia.
- Separación de responsabilidades.
- Código modular.
- Principios SOLID.
- Variables de entorno.

Entregables Esperados

El desarrollador deberá entregar:

Código Fuente

Repositorio GitHub funcional.

Base de Datos

- Script SQL.
- Modelo relacional.

Evidencias ETL

- Logs.
- Capturas.
- Reportes.

Documentación Técnica

- Instalación.
- Arquitectura.
- APIs.
- Dependencias.

Manual de Usuario

- Login.
- Uso dashboard.
- ETL.
- Reportes.

Diagramas

- Arquitectura.
- Flujo ETL.
- ERD.

Evidencia Machine Learning

- Dataset entrenado.
- Métricas.
- Resultados.

Criterios de Evaluación

Criterio	Peso
Arquitectura Backend	20%
Proceso ETL	25%
Analítica de Datos	15%
Machine Learning	15%
Frontend/Dashboard	10%
Seguridad	5%
Documentación	10%

Bonus Opcionales

Nivel Avanzado

Agregar:

- Dockerización.
- Despliegue Cloud.
- Celery + Redis.
- WebSockets.
- Predicción en tiempo real.
- IA generativa para análisis médico.
- Integración HL7/FHIR.

Resultado Esperado

El Aprendiz deberá construir una solución profesional capaz de:

- Procesar datos clínicos reales simulados.
- Automatizar ETL.
- Analizar información médica.
- Detectar pacientes críticos.
- Ejecutar modelos predictivos.
- Visualizar métricas clínicas.
- Generar valor real para el sector salud.

Este reto evalúa competencias integrales en:

- Desarrollo FullStack.
- Ingeniería de Datos.
- ETL.
- Machine Learning.
- Analítica de Datos.
- Arquitectura de Software.
- Backend escalable.
- Sistemas inteligentes aplicados al sector salud.

Fecha de entrega: 15 de junio de 2026

Modalidad Trabajo: Individual